SMART RESTAURANT MANAGEMENT SYSTEM

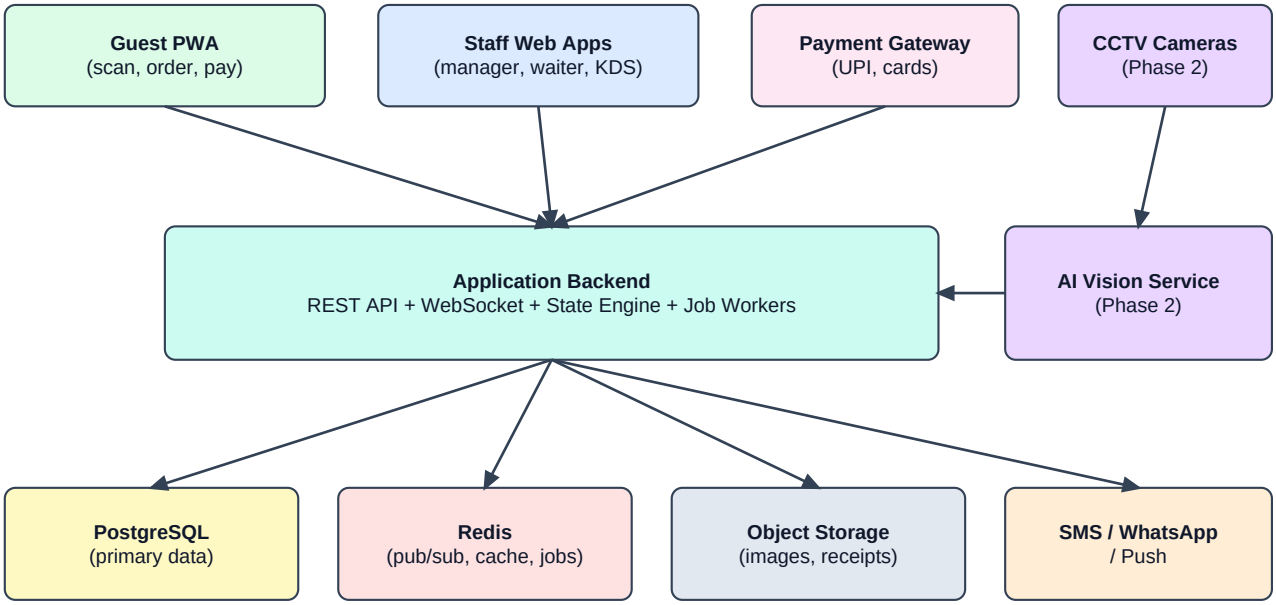
System Architecture and Tech Stack

Components, technologies, real-time, security and deployment

DOCUMENT ID	SRMS-07
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	How the system is built: components, recommended stack, real-time design, security and the Phase 2 AI pipeline.

1. Architecture Overview

A modular monolith is recommended for Phase 1: one deployable backend with clear internal modules, a single PostgreSQL database and Redis for real-time messaging. This is simple to build and operate, and the AI vision service is the only separate service, added in Phase 2.



2. Recommended Technology Stack

Layer	Choice	Why
Guest app	Next.js (React) PWA, Tailwind CSS	No install needed; fast on mobile; SEO and QR deep links work well
Staff apps	React (same codebase or separate routes), installable PWA	One skill set; works on tablets and phones; push notifications
Backend	Node.js with NestJS (TypeScript)	Modular structure matches our modules; strong typing; good WebSocket support
Real-time	Socket.IO over Redis adapter	Live floor map, kitchen tickets, alerts; auto-reconnect; rooms per restaurant and per table
Database	PostgreSQL 15+ with Prisma or TypeORM	Transactions for orders and payments; partial unique indexes; JSONB
Cache / queues	Redis with BullMQ	Timers for alert rules, retries, webhooks processing
Payments	Razorpay (UPI, cards, wallets) with webhook verification	India focus; UPI first; good docs and test mode
Notifications	Web Push, WhatsApp Business or SMS provider	Receipts, waitlist messages, staff alerts
Storage	S3-compatible bucket plus CDN	Menu images, QR PDFs, receipts
Auth	JWT access and refresh tokens, RBAC guard, argon2 hashing	Stateless scaling; role checks on every route
Hosting	Docker on a managed cloud (AWS, GCP or DigitalOcean), managed Postgres	Simple operations; backups handled
CI/CD and quality	GitHub Actions, ESLint, Jest, Playwright, Sentry	Automated testing and error tracking

Layer	Choice	Why
AI (Phase 2)	Python service, YOLO-family detector via ONNX/OpenCV, edge device optional	Real-time person/object detection; can run on a small GPU or edge box

3. Backend Module Layout

```
backend/src
  modules/
    session/      (QR, table sessions)
    menu/         (categories, items, modifiers)
    orders/       (cart, orders, items)
    kitchen/      (KDS queries, item status)
    billing/      (bills, tax, invoices)
    payments/     (gateway, webhooks, refunds)
    tables/       (tables, floors, state engine)
    alerts/       (rules, escalation)
    cleaning/     (tasks, queue)
    auth/         (staff, roles, guards)
    admin/        (settings, QR generation)
    reports/      (analytics queries)
    vision/       (Phase 2 event ingestion)
  realtime/       (socket gateway, event publisher)
  common/         (guards, filters, audit, utils)
  jobs/           (timers, schedulers)
```

4. Real-time Design

- Domain events (TableOccupied, OrderPlaced, ItemReady, AlertRaised, PaymentSucceeded, TableCleaned) are published after each database commit.
- Staff clients join a **restaurant room**; guest clients join a **session room** and only receive their own table events.
- On reconnect, clients fetch a fresh snapshot (floor map, open alerts, active orders) and then continue with live events, so nothing is missed.
- Redis pub/sub lets multiple backend instances broadcast consistently.
- Timers (idle sessions, delayed orders, unattended tables) run as scheduled jobs that check state and emit alert events.

5. State Engine Design

- A single module is the only code allowed to update `tables.status`.
- Input: events with actor and reason. The engine validates the transition against the allowed-transition table, writes the new status and the log row in one transaction, then publishes an event.
- Row-level locking on the table row avoids race conditions (two scans at once, payment and override at once).
- Phase 2 AI events use exactly the same input path with `actor_type = AI` and a confidence value.

6. Security Architecture

Area	Control
Transport	HTTPS only, HSTS, secure WebSocket
QR abuse	Long random table tokens, rate limiting per IP and token, session token binding, rotating QR if abused
Authentication	Hashed passwords and PINs, lockout, refresh token rotation, optional 2FA for manager/owner
Authorisation	Role guards on every endpoint; restaurant_id enforced in every query (tenant isolation)
Input	Server-side validation, parameterised queries, output encoding, file type and size limits
Payments	Server-calculated amounts, signed webhooks, idempotency keys, no card data stored

Area	Control
Secrets	Environment secrets in a vault or encrypted store; gateway keys encrypted at rest
Data protection	Encrypted backups, least-privilege database users, audit log
Camera data (Phase 2)	Encrypted stream URLs, edge processing, no face data, short snapshot retention

7. Deployment and Environments

Environment	Purpose	Notes
Local	Developer machines	Docker Compose: API, Postgres, Redis
Staging	Testing and demos	Gateway in test mode; seeded demo restaurant
Production	Live restaurants	Managed Postgres with backups, autoscaling API, monitoring and alerts

- CI pipeline: lint, unit tests, integration tests, build image, deploy to staging, smoke test, manual approval to production.
- Observability: structured logs, error tracking (Sentry), metrics and uptime checks, alert if payment webhook failures spike.
- Backups: daily full plus point-in-time recovery; restore drill every quarter.

8. Phase 2 Architecture: AI Camera Pipeline

1. Cameras stream RTSP video to an edge box in the restaurant (or a cloud GPU service).
2. The vision service samples frames (for example 2 to 5 per second), runs person/chair/table detection and counts people per table zone.
3. A small rules layer turns raw detections into events such as PERSON_SEATED, TABLE_CLEARED, LONG_UNATTENDED, QUEUE_COUNT with a confidence score.
4. Events are posted to the backend vision endpoint with a signed token.
5. The state engine applies high-confidence events and creates suggestions for low-confidence ones; manual actions always win.
6. Only counts, states and optional short-lived snapshots leave the edge device.